

Employee Management System

Project Report

A responsive full-stack employee directory and management platform.

STUDENT	Vedant Pokale
---------	---------------

BRANCH	ENCS
--------	------

SECTION	C
---------	---

SERIAL NUMBER	49
---------------	----

PROJECT	PeopleFirst — Employee Management System
---------	--

TECHNOLOGY	HTML · CSS · JavaScript · Node.js · Express · SQLite
------------	--

DATE	Wednesday, October 7, 2026
------	----------------------------

[GitHub: github.com/vedantpokale22/employee-management-system](https://github.com/vedantpokale22/employee-management-system)

01 · OVERVIEW

Executive Summary

PeopleFirst is a browser-based Employee Management System (EMS) for maintaining employee records and viewing essential workforce information. The application combines a responsive HTML, CSS, and JavaScript interface with a Node.js and Express REST API and a persistent SQLite database. Its primary workflow is employee management: users can create, search, view, update, filter, export, and delete employee records.

The system includes a dashboard with employee-focused summaries, department distribution, a recent employee list, and employee profile views. Vedant Pokale is represented as the sample logged-in system administrator and is not part of the seeded employee records.

This is a functional coursework/demo system. Attendance figures are illustrative; leave approval, performance reviews, payroll processing, authentication, and authorization are not implemented.

02 · CONTEXT

Introduction and Problem Statement

Employee information is often spread across spreadsheets and disconnected documents. Common tasks—finding a person, checking their position or department, updating employment information, and maintaining consistency—can become slow and error-prone.

PeopleFirst addresses this need with one employee directory backed by a structured database. A browser interface communicates with a server through JSON REST endpoints. The server validates employee data before saving it to SQLite, while the interface supports directory workflows across desktop and mobile screen sizes.

03 · GOALS

Aim and Objectives

Aim: design and implement a responsive Employee Management System that stores and manages employee and organizational information through a web interface.

- Centralize employee records and employment details in one directory.
- Support create, read, update, and delete (CRUD) operations.
- Search by name, email, employee ID, department, and job title.
- Filter by department, status, position, and joining date.
- Provide workforce counts and department distribution.
- Persist records in SQLite between application restarts.
- Validate inputs on the server and use parameterized database queries.
- Provide responsive table/card views, profiles, and CSV export.

04 · SCOPE

Project Scope

Included

- Employee dashboard, KPIs, chart, department distribution, and recent employee records.
- Directory table and card layouts with search, filters, CSV export, and actions.
- Employee profile and add/edit form for personal, employment, and contact information.
- Department and illustrative attendance views.
- SQLite initialization, schema migration, and sample records.

Not included

- Authentication, authorization, roles, and secure account management.
- Live attendance event storage, leave approvals, performance reviews, or payroll processing.
- Notifications, audit trail, cloud hosting, or backup administration.

These boundaries are intentional for the project scope. Incomplete workflows are identified in the interface rather than presented as live business data.

05 · IMPLEMENTATION

Technology Stack

Layer	Technology	Responsibility
Frontend	HTML5	Semantic application structure, tables and forms
Styling	CSS3	Dark enterprise theme, responsive layout, and states
Browser logic	Vanilla JavaScript	REST calls, rendering, filtering, and interactions
Runtime	Node.js 20+	Server-side JavaScript runtime
Backend	Express 4	Static files, REST routes, JSON, and error handling
Database	SQLite / sql.js	Persistent relational records via WebAssembly
Tooling	npm	Dependency installation and run scripts

The requested coursework stack permits Node.js/Express or Python/Flask and SQLite or MongoDB. This implementation uses Node.js, Express, and SQLite.

06 · ARCHITECTURE

System Architecture

```
Browser UI
  !" JSON over HTTP
Express REST API
  !" validate and normalize
Parameterized SQL via sql.js
  !"
.data/employees.db
```

Request flow: a browser action sends a JSON fetch request; Express validates input; the API binds values into SQL statements; sql.js updates the SQLite database and persists the database file; Express returns a JSON response; the UI refreshes the relevant view.

07 · MODULES

Main Features

Dashboard

Shows total, active, department, and on-leave employee counts; an employee overview chart; department distribution; recent employees; quick actions; and an explicitly illustrative attendance snapshot. Chart points are reconstructed from current employee start dates and are not historical snapshots.

Employee directory

Supports search, filters by department/status/position/joining date, employee table and card views, profile access, editing, deletion confirmation, and CSV export of matching records.

Employee records and profiles

Profiles organize contact and employment information into tabs. The add/edit form captures identity, work role, department, manager, joining date, employment type, salary, and optional address details. Generated IDs use a pattern such as EMP-009.

Other screens

Departments are derived from the employees in SQLite. Attendance percentages are demo-only. Leave, performance, and payroll screens explain that those workflows have not been implemented.

08 · DATABASE

Database Design

The application uses one primary relational table: employees. The schema includes an internal auto-increment ID, a unique employee code, employee name and email, phone, department, job title, joining date, date of birth, employment type, manager, address fields, salary, status, and created/updated timestamps.

Data group	Fields / rules
Identity	id (primary key), employee_code (unique), first_name, last_name
Contact	email (case-insensitive unique), phone, address, city, state, country
Employment	department, job_title, manager, start_date, employment_type, status
Compensation	salary: required, numeric and non-negative
Metadata	created_at and updated_at timestamps

The database module creates the table on first run and migrates existing files to add newer profile columns. Starter records are inserted only when the employee table is empty. The database file is written under .data/employees.db and excluded from Git.

09 · BACKEND API

REST API Endpoints

Method	Endpoint	Purpose
GET	/api/employees	List employees; accepts search, department, status

GET	/api/employees/:id	Retrieve one employee by internal ID
POST	/api/employees	Create a validated employee record
PUT	/api/employees/:id	Update an employee record
DELETE	/api/employees/:id	Delete an employee record
GET	/api/dashboard	Return summary totals and department counts

Typical responses include 200 OK, 201 Created, and 204 No Content. Errors use 400 Bad Request for invalid input, 404 Not Found for missing records/routes, and 409 Conflict for duplicate email or employee ID.

10 · SAFEGUARDS

Validation, Security, and Privacy

- Required field, format, and maximum-length checks are enforced server-side.
- Email addresses are normalized to lowercase; statuses and employment types use allowlists.
- Dates must use an ISO-style format; salary must be numeric, finite, non-negative, and within the configured limit.
- Employee IDs and emails are unique; duplicate records return conflict responses.
- SQL user values are bound as parameters rather than concatenated into queries.
- Frontend rendering escapes employee-provided strings before inserting them into markup.

The demo has no login or access control. Home address, date of birth, email, and salary can be sensitive personal data. Do not publish real employee information until authentication, authorization, privacy review, secure hosting, and suitable safeguards are in place.

11 · USER EXPERIENCE

Interface and Accessibility

The dark navy interface uses restrained purple accents, subtle borders, clear typography, and responsive panels. Desktop users receive a fixed sidebar; tablet and phone layouts use a drawer. Directory records can be shown as a table or as cards.

- Semantic labels and headings, named controls, and dialog roles.
- Visible keyboard focus indicators and accessible status/toast announcements.
- Escape closes menus/dialogs; Ctrl/Cmd+K focuses global search.
- Reduced-motion preferences are respected.
- Responsive layouts stack dashboard content and employee cards at narrow widths.

12 · RUNNING THE APPLICATION

Installation and Operation

Step	Action
1	Install Node.js 20+ and npm.
2	Open the Employee Management System project directory in a terminal.
3	Run npm install to install dependencies.

4	Run <code>npm run dev</code> for development, or <code>npm start</code> for a normal run.
5	Open <code>http://localhost:3000</code> in a browser.
6	Press <code>Ctrl+C</code> in the server terminal to stop the process.

Do not start a second server on the same port while the first process is still running. The generated SQLite file is local to the machine unless a separate hosting/synchronization system is configured.

13 · VERIFICATION

Testing and Results

Check	Observed result
Node syntax checks	Server, database, and browser scripts passed.
Employee listing and dashboard	8 records; 6 active employees; 6 departments.
Search and department/status filters	Matching records returned in smoke checks.
Employee ID search and duplicate checks	Search worked; duplicate ID was rejected.
Create, update, delete API	Passed; temporary test records were removed.
Profile and tab navigation	Employee details and profile tabs rendered.
Table/card switch	Both layouts displayed the employee set.
Responsive layout	No document-wide overflow in tested browser view.

These are implementation-time manual smoke checks, not a comprehensive automated test suite. Counts represent the current demonstration database and may change when records are edited.

14 · SOURCE LAYOUT

Project Structure

Path	Purpose
<code>server.js</code>	Express server, API routes, validation, startup.
<code>src/database.js</code>	SQLite schema, migrations, sample data, persistence adapter.
<code>public/index.html</code>	Dashboard, directory, profile, and form markup.
<code>public/app.js</code>	Browser API calls, rendering, profiles, filters, export.
<code>public/styles.css</code>	Core dark theme and responsive dashboard styling.
<code>public/responsive.css</code>	Drawer navigation and small-screen layout.
<code>README.md</code>	Quick setup, feature summary, and API guide.
<code>REPORT.md</code>	Full coursework project report source.

15 · FUTURE WORK

Limitations and Enhancements

- Implement login, secure sessions, authorization, and role-based permissions.
- Add automated API and UI tests, pagination, and server-side sorting.
- Add audit logs, backup/restore, production logging, and secure deployment.
- Implement attendance events, shifts, leave requests, and approval workflows.
- Add performance-review records and a documented payroll workflow/integration.
- Use a real historical workforce data model for chart trends.
- Replace demonstration seed data and support placeholders before real use.

16 · CONCLUSION

Conclusion

PeopleFirst demonstrates a full-stack foundation for employee record management. It connects a responsive vanilla JavaScript interface to an Express REST API and persistent SQLite storage. The system validates employee data and supports directory, profile, search, filter, and export workflows. The project is suitable as a coursework demonstration; the stated security, privacy, and workflow limitations should be addressed before any production use.

17 · REFERENCES

Project References and Submission Details

GitHub source repository: <https://github.com/vedantpokale22/employee-management-system>

Project setup and API details are documented in README.md. Main backend and database implementations are in server.js and src/database.js.

Submission field	Details
Student name	Vedant Pokale
Branch	ENCS
Section	C
Serial number	49
Course / subject	MDM Backend Development
Institution	Ramdeobaba University
Faculty / guide	Sameer Tembhurney

18 · APPLICATION SCREENS

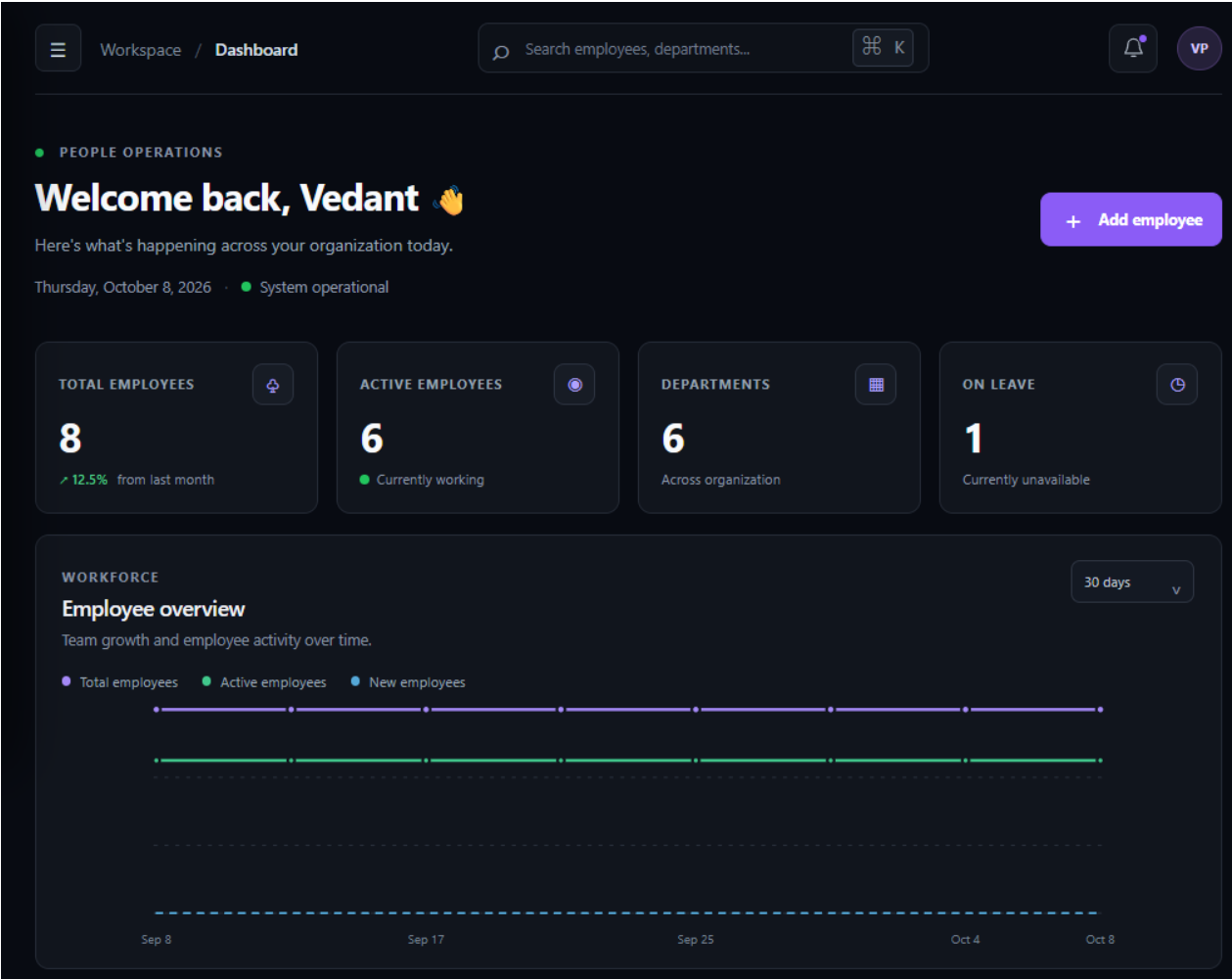
Dashboard Screenshots

The dashboard is shown in four consecutive captures so its workforce metrics, charts, employee records, attendance snapshot, and activity feed remain readable in the PDF.

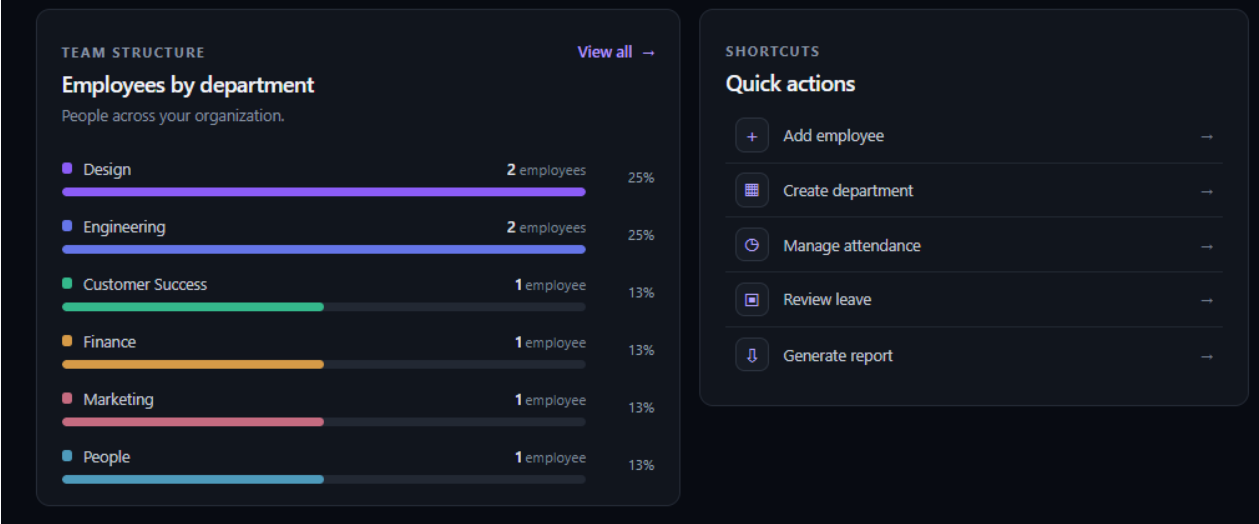
Complete dashboard view



Dashboard overview and workforce metrics



Employee overview and department distribution



Recent employee records and quick actions

PEOPLE DIRECTORY

Recent employees

Your latest employee records.

[View all](#) →

EMPLOYEE	EMPLOYEE ID	DEPARTMENT	POSITION	STATUS	JOIN DATE	
AL Andi Lane andi.lane@example.com	EMP-008	Design	UX Researcher	Active	May 20, 2024	...
DC Drew Cano drew.cano@example.com	EMP-007	Customer Success	Customer Success Lead	Active	Sep 4, 2023	...
NC Natali Craig natali.craig@example.com	EMP-006	Finance	Financial Analyst	Inactive	Jun 27, 2022	...
CW Candice Wu candice.wu@example.com	EMP-005	Engineering	Software Engineer	Active	Feb 12, 2024	...
DW Demi Wilkinson demi.wilkinson@example.com	EMP-004	People	HR Business Partner	Active	Nov 9, 2020	...

Attendance snapshot and recent activity

